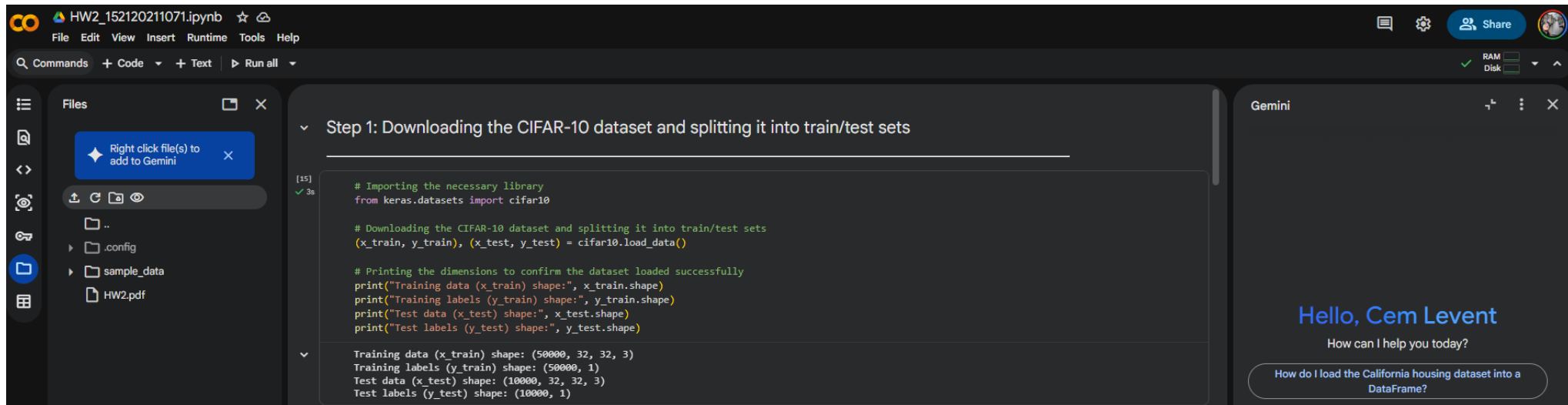


HW2 REPORT



The screenshot displays a Jupyter Notebook environment with a dark theme. The top bar shows the notebook name 'HW2_152120211071.ipynb' and standard menu options. The left sidebar contains a 'Files' panel with a file explorer showing a directory structure with files like '.config', 'sample_data', and 'HW2.pdf'. The main area is titled 'Step 1: Downloading the CIFAR-10 dataset and splitting it into train/test sets'. It contains a code cell with the following Python code:

```
[15]: # Importing the necessary library
      from keras.datasets import cifar10

      # Downloading the CIFAR-10 dataset and splitting it into train/test sets
      (x_train, y_train), (x_test, y_test) = cifar10.load_data()

      # Printing the dimensions to confirm the dataset loaded successfully
      print("Training data (x_train) shape:", x_train.shape)
      print("Training labels (y_train) shape:", y_train.shape)
      print("Test data (x_test) shape:", x_test.shape)
      print("Test labels (y_test) shape:", y_test.shape)
```

Below the code cell, the output is displayed, showing the shapes of the loaded data and labels:

```
Training data (x_train) shape: (50000, 32, 32, 3)
Training labels (y_train) shape: (50000, 1)
Test data (x_test) shape: (10000, 32, 32, 3)
Test labels (y_test) shape: (10000, 1)
```

On the right side of the interface, there is a 'Gemini' chat panel. It displays a greeting 'Hello, Cem Levent' and a prompt 'How can I help you today?'. A suggested question is visible: 'How do I load the California housing dataset into a DataFrame?'.

Within the scope of this report, a K-Nearest Neighbors (K-NN) classification algorithm has been implemented to classify images from the CIFAR-10 dataset. The primary objective of this study is to understand the underlying mechanics of pattern recognition by developing a Custom Similarity measure from scratch, rather than relying on standard pre-built metric libraries.

TASK 1: DATA PREPARATION AND VECTORIZATION

For this task, the CIFAR-10 dataset, consisting of 50,000 training images and 10,000 test images, was downloaded and processed:

- **Vector Transformation:** The original image matrices, which have dimensions of $32 \times 32 \times 3$ (representing RGB channels), were flattened into 1-dimensional vectors of size 1×3072 . This vectorization step is crucial as the K-NN algorithm computes spatial metrics across flat feature arrays rather than multi-dimensional image grids.

```
Training data (x_train) shape: (50000, 32, 32, 3)
Training labels (y_train) shape: (50000, 1)
Test data (x_test) shape: (10000, 32, 32, 3)
Test labels (y_test) shape: (10000, 1)
```

```
Vectorized training data (x_train) shape: (50000, 3072)
Vectorized test data (x_test) shape: (10000, 3072)
```

TASK 2: SIMILARITY METRIC IMPLEMENTATION

To determine the proximity of a given test sample to the training data, a mathematical model was developed:

- **L2 Distance Calculation:** The Euclidean distance (L2 norm) between the test vector and the training vectors was computed. During this matrix operation, image pixel arrays were explicitly cast from **uint8** to **float32** data types. This crucial step prevents "Integer Underflow" errors, ensuring that negative pixel differences are calculated accurately rather than overflowing into large positive integers.
- **Distance to Similarity Conversion:** The assignment instructions required identifying the "most similar" items by sorting scores in descending order. Therefore, the calculated **L2 distance (d)** was inverted into a **similarity score (S)** using the mathematical formula $S = \frac{1}{1 + d}$. Consequently, images with the shortest distance yielded the highest similarity scores.

TASK 3: K-NN CLASSIFICATION AND MAJORITY VOTING

The core classification algorithm was executed by comparing specific test samples against the entire 50,000-image training set.

- **Sorting and Selection:** The calculated similarity scores were sorted in descending order (e.g., from highest decimal value to lowest). Based on a predefined k parameter (e.g., k=5 or k=10), the top k nearest neighbors were isolated.
- **Majority Vote:** The class labels of these nearest neighbors were analyzed, and the test sample was assigned to the class with the highest frequency (mode) among them.

```
***  
_____  
CLASSIFICATION STATISTICS  
_____  
[INPUT DETAIL] K Parameter: 5  
  
[SORTED NEAREST NEIGHBORS]  
Neighbor 1 -> Similarity Score: 0.00040521 | Class: deer  
Neighbor 2 -> Similarity Score: 0.00038789 | Class: bird  
Neighbor 3 -> Similarity Score: 0.00038733 | Class: bird  
Neighbor 4 -> Similarity Score: 0.00038583 | Class: deer  
Neighbor 5 -> Similarity Score: 0.00038559 | Class: bird  
  
[VOTE DISTRIBUTION (MAJORITY VOTE)]  
- bird: 3 votes  
- deer: 2 votes  
_____  
[OUTPUT RESULT] Predicted Class: >> BIRD <<
```

In conclusion, this assignment successfully demonstrated the fundamental mechanics of a K-Nearest Neighbors classifier by building it entirely from scratch. By transforming 3D image matrices into flat 1D vectors, the algorithm effectively evaluated spatial relationships across the **CIFAR-10 dataset**. The core objective of implementing a similarity metric was achieved by calculating the **L2 (Euclidean)** distance and mathematically inverting it to prioritize the closest neighbors.

Furthermore, a critical aspect of this implementation was managing data constraints; specifically, converting the native **uint8** pixel arrays to **float32** structures. This prevented integer underflow during vector subtraction, ensuring absolute mathematical accuracy in the proximity calculations. Ultimately, the integration of a majority voting mechanism correctly categorized the test samples, validating the structural integrity and robustness of traditional pattern recognition algorithms.